

CampusEcho: Anonymous & Accountable Feedback System

CS455 Project: Requirements and Architecture Document

Kishore Senthilkumar	230566
B. Vaishnavi	230295
M. Amara Gayathri	230624
Reddi Pallavi	230850
Poorvie Sadagopan	230759
Vundela Obula Reddy	231178

September 26, 2026

Contents

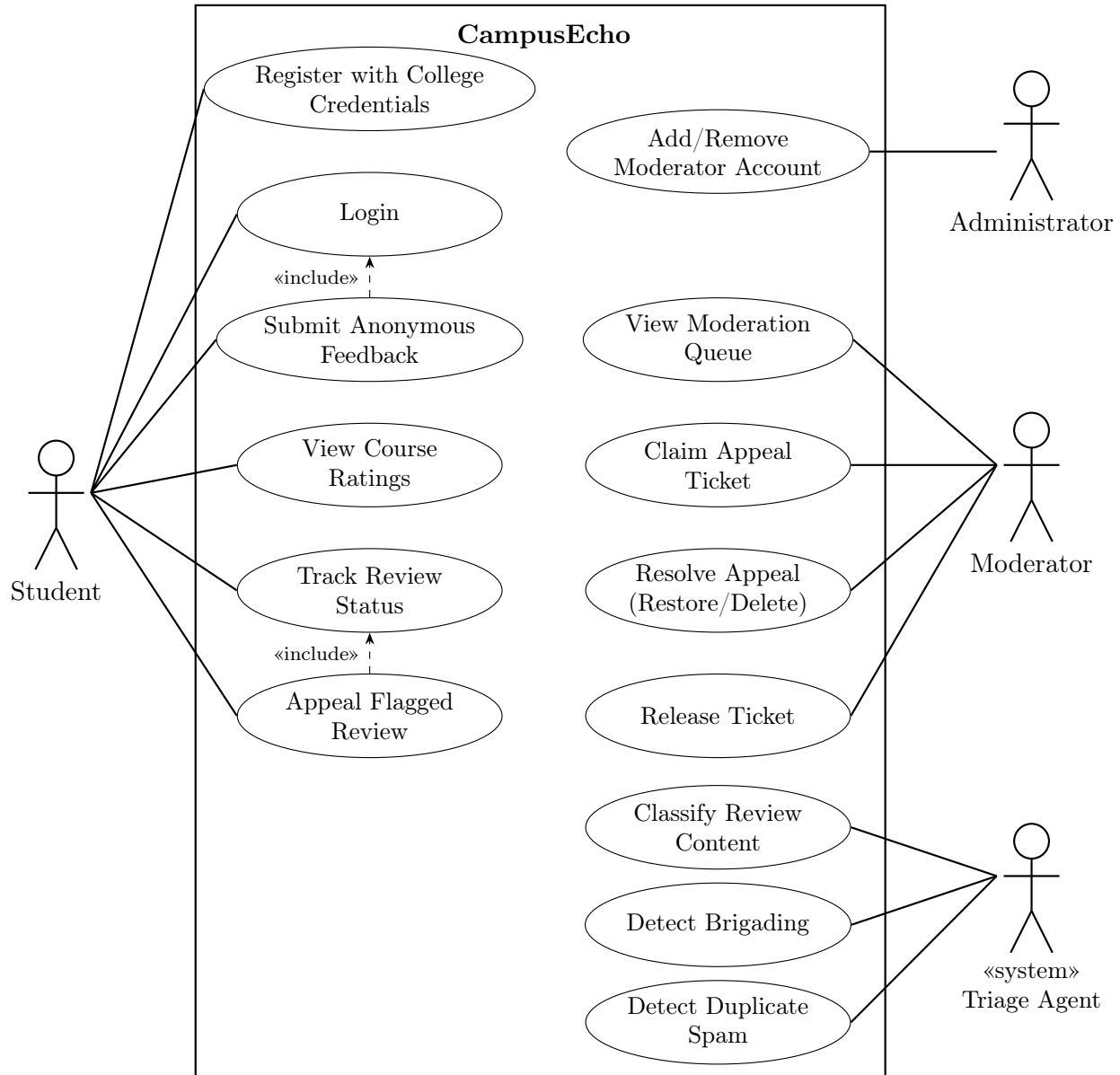
1	Requirements Specification	3
1.1	Use-Case Diagram	3
1.2	User Stories	3
1.3	Functional Requirements	5
1.4	Non-Functional Requirements	5
1.5	Initial Requirements Traceability Matrix	6
2	Architecture and Design	8
2.1	Component Diagram	8
2.2	Sequence Diagrams	9
2.3	Design-Pattern Mapping	11
2.4	Architecture Alternatives and Rejected Decisions	12
3	Project Management Setup	13
3.1	Jira Project and Initial Backlog	13
3.2	Jira-GitHub Integration	14
3.3	Initial Sprint Plan	15
4	AI Engineering Log for Requirements and Architecture Activities	16
4.1	AI-Assisted Requirements Refinement	17
4.1.1	Objective	17
4.1.2	Prompt / Interaction	17
4.1.3	LLM Output	18
4.1.4	Team Evaluation	18
4.1.5	Changes Made	18
4.1.6	Final Decision	18
4.2	AI-Assisted Architecture Generation	19

4.2.1	Objective	19
4.2.2	Prompt / Interaction	19
4.2.3	LLM Output	19
4.2.4	Team Evaluation	19
4.2.5	Final Architecture	20

1 Requirements Specification

1.1 Use-Case Diagram

Figure 1.1 shows the use cases of CampusEcho and the actors who interact with them. The Student and Moderator are primary human actors. The Triage Agent is a secondary system actor that processes submitted reviews asynchronously and never has access to student identities.



1.2 User Stories

Story 1: Account Creation and Role-Based Access

Ananya registers using her college credentials to receive a student session token for submitting feedback and viewing ratings. Meanwhile, an administrator provisions a moderator account for a staff member, issuing an session token that grants access to view and manage the moderation queue.

Story 2: Submitting Course Feedback

Riya is a student who wants to review her Operating Systems course after the mid-semester exam. She logs in, opens the feedback page, selects the course and instructor, gives a star rating and writes a short review. She clicks “Submit” and the system immediately confirms that her feedback has been received and shows her an anonymous reference code, which she can save to track her review later. Her name is not stored with the review. In the background, the triage agent checks the review, and since it contains no abusive content, it is published on the course page within a few minutes.

Story 3: Viewing Course Ratings

Arjun is choosing his electives for next semester. He opens CampusEcho, searches for a course and sees its average rating along with the published reviews. Reviews that have been flagged, are under appeal or have been deleted are not shown. He can sort reviews by most recent or by rating.

Story 4: Appealing a Flagged Review

Meera wrote a strongly critical review of a course, but the triage agent flagged it as abusive and it was not published. She enters her anonymous reference code and sees that her review’s status is “Flagged”, along with the reason. She believes her review is harsh but fair, so she clicks “Appeal” and adds a short explanation. The system raises an appeal ticket, places it in the moderation queue and changes her review’s status to “Under Appeal”. She can appeal a given review only once.

Story 5: Claiming an Appeal Ticket

Rahul is a moderator. He opens the moderation queue, which lists all unclaimed appeal tickets, oldest first. He clicks “Claim” on a ticket, and the system locks it to him and moves it to his “My Tickets” list. If another moderator has claimed the same ticket a moment earlier, Rahul sees the message “This ticket has already been claimed”, and the queue refreshes. A ticket can never be assigned to two moderators.

Story 6: Resolving an Appeal

Rahul opens a ticket he has claimed and reads the original review, the reason it was flagged and the student’s explanation. If he decides the review is acceptable, he clicks “Restore” and the review is published. If he decides it breaks the guidelines, he clicks “Delete” and the review is permanently removed. Either way, the ticket is closed, and the student sees the final outcome when they check their reference code. Only the moderator who claimed the ticket can resolve it.

Story 7: Releasing an Abandoned Ticket

Priya, a moderator, claims a ticket but is called away before resolving it. She can click “Release” to return it to the queue. If she does not, the system automatically releases the lock after 30 minutes, so the ticket becomes available to other moderators and is not stuck forever.

Story 8: Detecting a Brigading Attack

A group of students tries to lower a professor’s rating by posting 20 near-identical negative reviews within ten minutes. The triage agent notices that the submission velocity is above 15 reviews per ten minutes and that the reviews are more than 75% similar to each other, so it flags the whole

batch and none of the reviews are published. In contrast, after the end-semester exam, 30 students review the same course within ten minutes, but their reviews are all different. The agent sees the high velocity but low similarity and publishes them normally.

Story 9: Detecting Repeated Spam

A student submits the same long review for a course five times over an afternoon. The submission rate is normal, but the triage agent detects that the reviews are near-duplicates. It publishes the first one and flags the rest as duplicate spam, without triggering a brigading alert.

1.3 Functional Requirements

FR-1 The system shall authenticate student sign-ups and admin provisioned moderators to issue role based session tokens.

FR-2 The system shall provide login with two roles, Student and Moderator, and restrict moderation features to moderators.

FR-3 A student shall be able to submit a review for a course (a 1–5 rating and text). The review shall be stored without the student’s identity, and the student shall receive an anonymous reference code.

FR-4 The system shall display each course’s average rating and its published reviews only.

FR-5 A student shall be able to check a review’s status and flag reason using its reference code.

FR-6 The triage agent shall asynchronously classify each review and either publish it or flag it as abusive, recording the reason.

FR-7 The triage agent shall flag near duplicate reviews for the same course as spam.

FR-8 The system shall flag a batch of reviews for a course as brigading when the velocity $V > 15$ reviews per 10 minutes and the average semantic similarity $S > 75\%$, counting only reviews longer than 5 words.

FR-9 A student shall be able to appeal a flagged review once, with a short explanation, which creates a ticket in the moderation queue.

FR-10 Moderators shall be able to view all unclaimed tickets, oldest first.

FR-11 A moderator shall be able to claim a ticket atomically, so that exactly one moderator succeeds when several claim it at once.

FR-12 A claimed ticket shall return to the queue if the moderator releases it or does not resolve it within 30 minutes.

FR-13 Only the moderator holding a ticket shall be able to resolve it by restoring the review (published) or deleting it (permanently removed).

1.4 Non-Functional Requirements

NFR-1 Anonymity. The review collection shall contain no field that can identify a student. Student identities shall be stored only in a separate collection. *Verified by a schema review of the review collection.*

Transition	Requirement
Submitted $\rightarrow$ Published / Flagged	FR-6, FR-7, FR-8
Flagged $\rightarrow$ Under Appeal	FR-9
Under Appeal $\rightarrow$ Locked	FR-11
Locked $\rightarrow$ Under Appeal (release)	FR-12
Locked $\rightarrow$ Restored / Deleted	FR-13

Table 1: Lifecycle transitions and the requirements that trigger them

Case	Values	Expected	Requirement
Post-exam organic surge	$V = 30, S = 30\%$	Published	FR-7 not triggered
Slow drip spam	$V < 15, S = 95\%$	Micro-flagged	FR-6
Coordinated attack	$V = 20, S = 85\%$	Macro-flagged	FR-7

Table 2: Proposal boundary cases and the requirements that govern them

NFR-2 Agent isolation. The triage agent’s database account shall have no read access to the identity collection. *Verified by attempting a query on the identity collection with the agent’s credentials.*

NFR-3 Concurrency correctness. When 50 concurrent claim requests are sent for the same ticket, exactly one shall succeed, in every test run. *Verified by a concurrent-claim load test.*

NFR-4 Responsiveness. Submitting a review shall return a response within 1 second, independent of AI service latency. *Verified by a response-time test with a deliberately slowed AI stub.*

NFR-5 Triage latency. Under normal load, a review shall be triaged within 60 seconds of submission. *Verified by comparing submission and triage timestamps in the logs.*

NFR-6 Fault tolerance. If the AI service fails or times out, the review shall remain in the *Submitted* state and be retried; it shall never be published without triage. *Verified by a test with the AI service disabled.*

NFR-7 Classification quality. On a labelled test set of 50 reviews that includes harsh but legitimate criticism, the agent shall wrongly flag at most 10% of legitimate reviews. *Verified by evaluating the agent on the labelled test set.*

NFR-8 Security. Passwords shall be stored hashed, all role checks shall be enforced by the backend, and all traffic shall use HTTPS. *Verified by code review and role-access tests.*

1.5 Initial Requirements Traceability Matrix

A requirements traceability matrix links each requirement to where it came from and to the work that delivers it, so no requirement is lost or left unjustified. Table 3 traces every functional and non-functional requirement to its source, the use case it supports and its Jira issue.

ID	Requirement	Source	Use Case	Jira
FR-1	Student registration, admin-provisioned moderators, role-based tokens	User Story 1: Account Creation and Role-Based Access	All	SCRUM-1
FR-2	Role-based login for students and moderators	Proposal: User Roles	All	SCRUM-2
FR-3	Anonymous review submission with a reference code	User Story 2: Submitting Course Feedback	Submit Anonymous Feedback	SCRUM-3
FR-4	View average rating and published reviews	User Story 3: Viewing Course Ratings	View Course Ratings	SCRUM-4
FR-5	Check review status using the reference code	User Story 4: Appealing a Flagged Review	Track Review Status	SCRUM-5
FR-6	Classify review content as publish or flag	Proposal: Agentic AI Integration (micro level)	Classify Review Content	SCRUM-6
FR-7	Flag near-duplicate reviews as spam	User Story 9: Detecting Repeated Spam	Detect Duplicate Spam	SCRUM-7
FR-8	Flag brigading when $V > 15$ and $S > 75\%$	Proposal: Agentic AI Integration (macro level)	Detect Brigading	SCRUM-8
FR-9	Appeal a flagged review once	User Story 4: Appealing a Flagged Review	Appeal Flagged Review	SCRUM-9
FR-10	View unclaimed tickets, oldest first	User Story 5: Claiming an Appeal Ticket	View Moderation Queue	SCRUM-10
FR-11	Claim a ticket atomically	Proposal: Shared Resource and Concurrency Challenge	Claim Appeal Ticket	SCRUM-11
FR-12	Release a ticket, or expire its lock after 30 minutes	User Story 7: Releasing an Abandoned Ticket	Release Ticket	SCRUM-12
FR-13	Restore or delete a claimed review	User Story 6: Resolving an Appeal	Resolve Appeal	SCRUM-13
NFR-1	No identifying data in the review collection	Proposal Risk: Decoupling authentication from feedback	All	SCRUM-14
NFR-2	Triage agent has no access to identities	Proposal: Agentic AI Integration (prohibited boundary)	Classify Review Content	SCRUM-15
NFR-3	Exactly one of 50 concurrent claims succeeds	Proposal Risk: Concurrency correctness under load	Claim Appeal Ticket	SCRUM-16
NFR-4	Submission responds within 1 second	Proposal: Agentic AI Integration (non-blocking triage)	Submit Anonymous Feedback	SCRUM-17
NFR-5	Triage completes within 60 seconds	User Story 2: Submitting Course Feedback	Classify Review Content	SCRUM-18
NFR-6	Retry triage if the AI service fails	Proposal Risk: LLM misclassification	Classify Review Content	SCRUM-19
NFR-7	At most 10% of legitimate reviews wrongly flagged	Proposal Risk: LLM misclassification	Classify Review Content	SCRUM-20
NFR-8	Hashed passwords, backend role checks, HTTPS	Proposal: User Roles	All	SCRUM-21

Table 3: Initial requirements traceability matrix

2 Architecture and Design

2.1 Component Diagram

Figure 1 illustrates the high-level modular architecture of CampusEcho. The system is decoupled into three primary tiers to ensure separation of concerns and maintainability.

- **Presentation Layer (React.js):** Contains the Student Dashboard (for submission, tracking, and appeals) and the Moderator Dashboard (for active queue management and ticket resolution).
- **Application Layer (Go Monolith):** Hosts the REST API Gateway, the Ticket Management Service for handling state transitions, role based authentication service for users and the AI Triage Worker Pool which manages asynchronous calls to the external LLM (Claude/AWS Bedrock).
- **Data Layer (MongoDB):** Stores isolated collections for user credentials, feedback payloads, and moderation tickets, utilizing atomic document updates to strictly enforce resource locking and privacy boundaries.

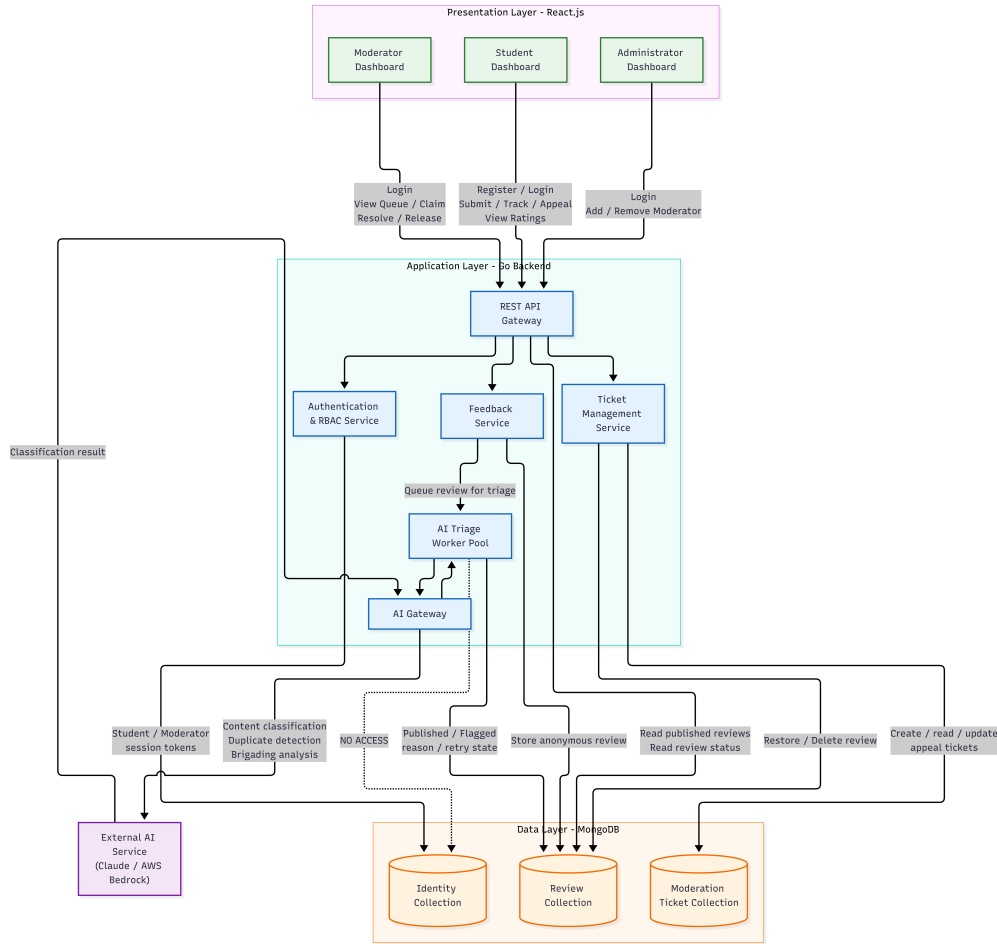


Figure 1: Component architecture of CampusEcho

2.2 Sequence Diagrams

To demonstrate the system's dynamic behavior, we focus on the two most critical workflows involving asynchronous processing and concurrency.

1. Asynchronous AI Triage Workflow:

When a student submits feedback, the Go backend stores it anonymously and returns a reference code without waiting for AI processing. The AI Triage Worker Pool then asynchronously classifies the review, detects duplicate spam, and evaluates brigading using velocity (V) and similarity (S). Brigading is flagged when $V > 15$ reviews per 10 minutes and $S > 75\%$, considering reviews longer than five words. Reviews are then *Published* or *Flagged* with the reason recorded. If the AI service fails or times out, the review remains *Submitted* and is retried.

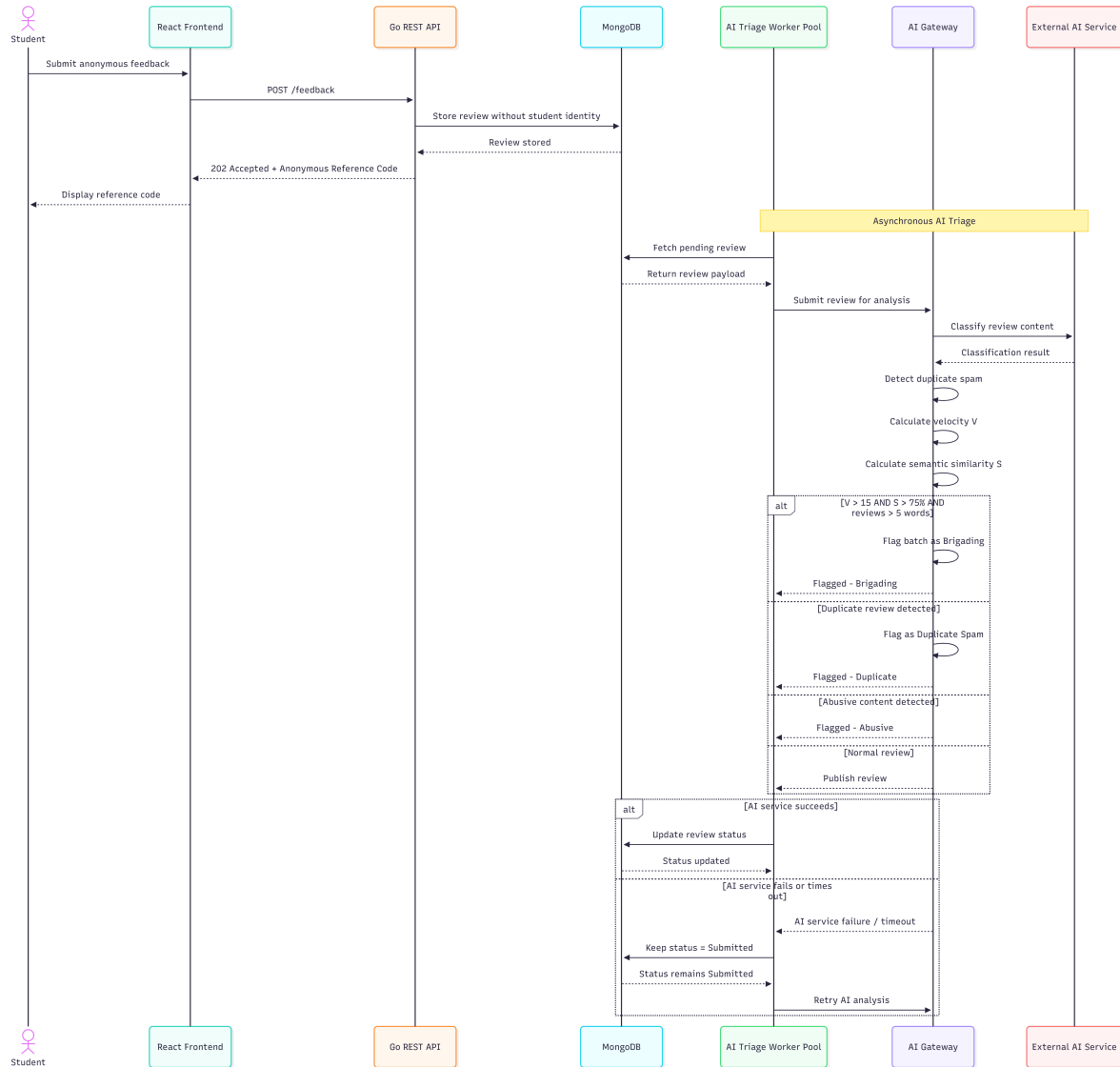


Figure 2: Sequence Diagram for the Asynchronous AI Triage Workflow

2. Concurrent Ticket Claiming Workflow:

When multiple moderators attempt to claim the same appeal ticket simultaneously, the backend performs an atomic claim operation in MongoDB. The first request successfully changes the ticket state from *Under Appeal* to *Locked*, while concurrent requests fail the state check and receive a **409 Conflict**. This ensures that exactly one moderator holds the ticket.

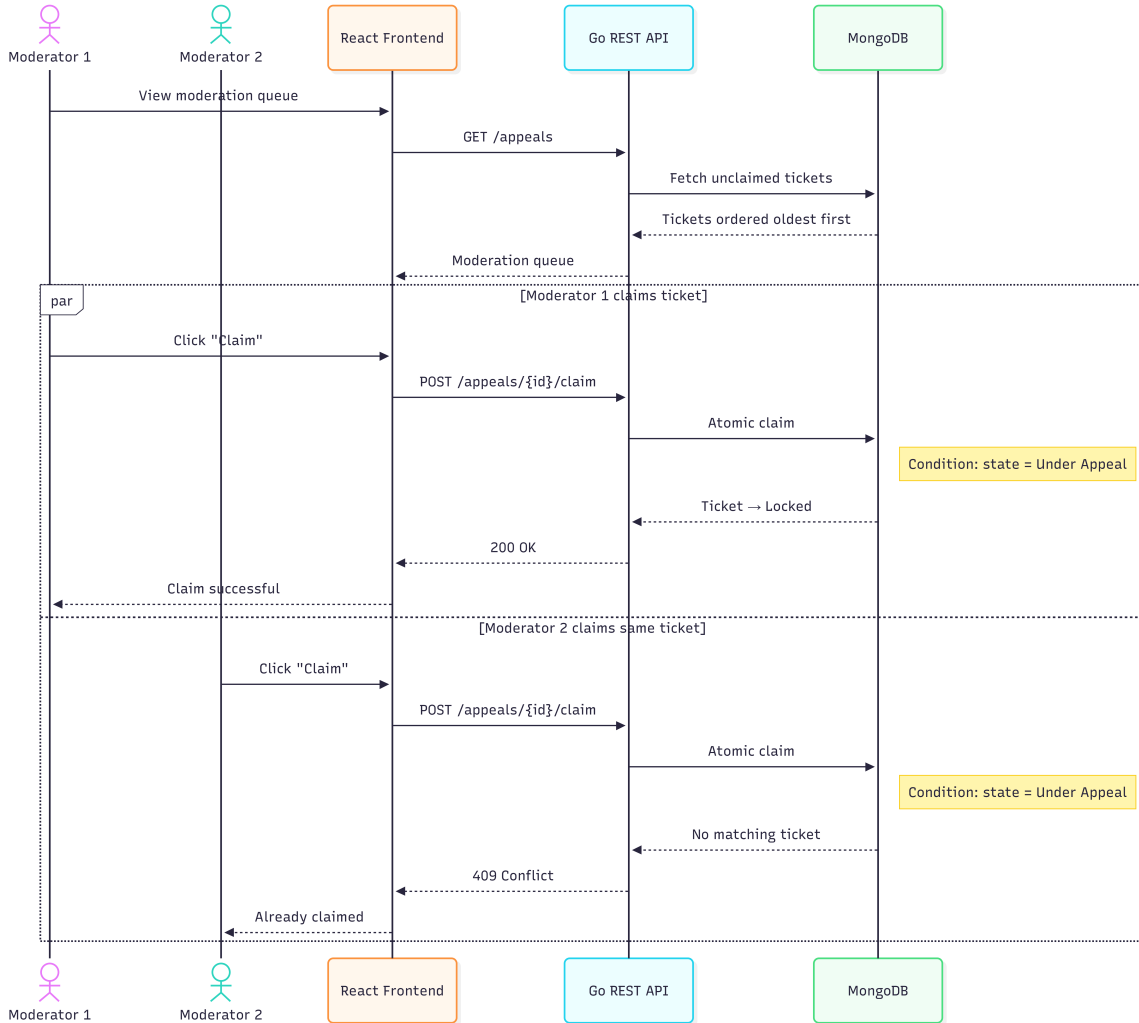


Figure 3: Sequence Diagram for Concurrent Appeal Ticket Claiming

3. Appeal Resolution and Ticket Lifecycle:

A student can appeal a flagged review once by providing a short explanation, creating a moderation ticket. After a moderator claims the ticket, it enters the *Locked* state. The moderator can either restore the review, permanently delete it, or release the ticket. A locked ticket that is released or remains unresolved for 30 minutes returns to the *Under Appeal* state.

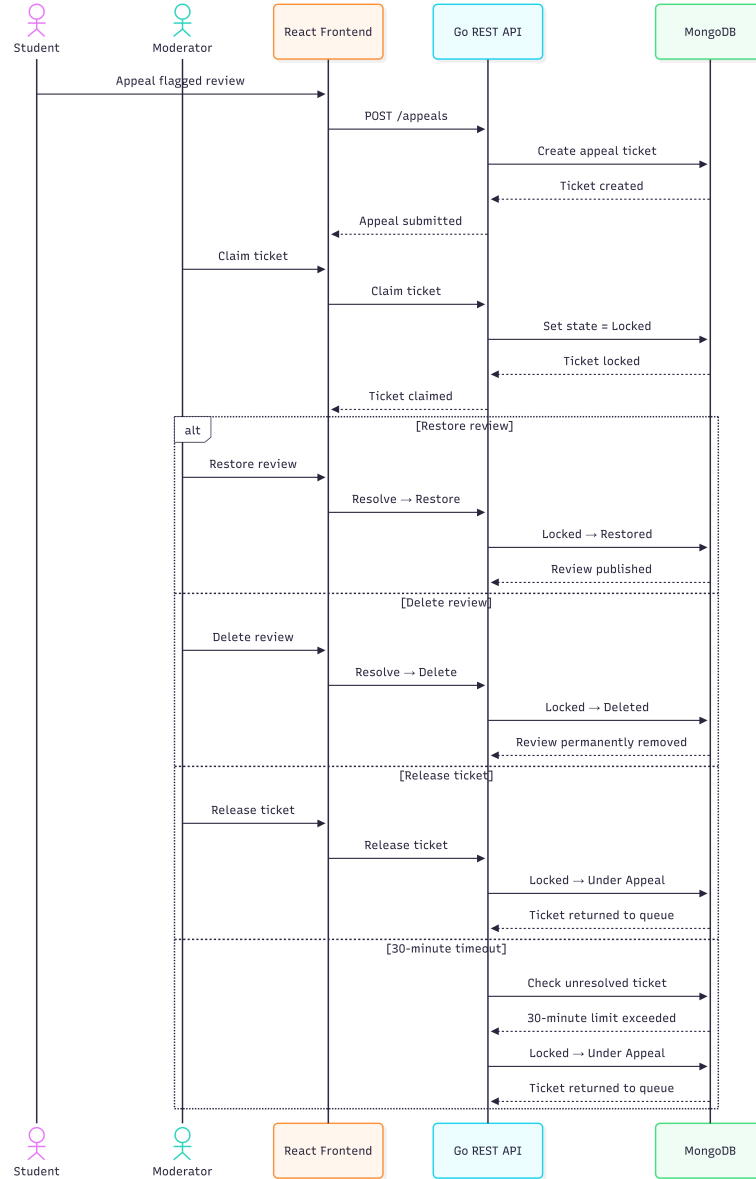


Figure 4: Sequence Diagram for Appeal Resolution and Ticket Lifecycle

2.3 Design-Pattern Mapping

The system architecture leverages the following established design patterns to address our core engineering constraints:

- **State Pattern:** Applied to the core transactional workflow of the feedback object. The feedback transitions through concrete states (*Submitted* → *Published/Flagged* → *Appealed* → *Locked* → *Restored/Deleted*). Operations behave differently depending on the current state (e.g., a ticket can only be claimed if it is currently in the *Appealed* state).
- **Worker Pool Pattern:** Utilized in the Go backend to manage the Autonomous Triage Agent. A fixed number of Goroutines operate as a pool to process incoming feedback submissions

asynchronously, preventing API rate limits with the external LLM and ensuring the main HTTP server threads remain unblocked under heavy load.

- **Singleton Pattern:** Implemented for the MongoDB connection client and the AI API client in the Go backend. This ensures that the system reuses a single, thread-safe connection pool across all concurrent incoming requests, optimizing resource utilization.
- **Strategy Pattern:** Used within the AI Gateway to dynamically swap evaluation criteria. The system applies a *Micro-Level Strategy* (content analysis for hate speech) and a *Macro-Level Strategy* (velocity and semantic similarity calculations for brigading) to determine the appropriate flagging behavior.

2.4 Architecture Alternatives and Rejected Decisions

To ensure a robust and maintainable system, we evaluated several architectural approaches for our core challenges. The following details three major engineering decisions, including the alternatives considered, the trade-offs, and our final rationale.

Decision 1: Concurrency and Distributed Locking

- **Considered Alternative:** Utilizing a dedicated Redis cluster for distributed locking (e.g., Redlock algorithm) to manage the moderation queue.
- **Trade-offs & Rationale:** Redis provides exceptionally fast in-memory locking. However, it introduces an additional infrastructural dependency and the risk of synchronization state mismatches between the Redis lock and the MongoDB ticket data.
- **Final Decision:** We opted to use MongoDB’s native atomic operations (specifically `findOneAndUpdate` with atomic state filters) to lock tickets. This ensures strict consistency within a single data-store, satisfying our concurrency constraints while minimizing deployment complexity.

Decision 2: Anonymity and Privacy Layer

- **Considered Alternative:** Implementing an anonymous credential system using Zero-Knowledge Proofs (zk-SNARKs) to cryptographically guarantee that the backend never links student authentication to the feedback payload.
- **Trade-offs & Rationale:** While cryptographic anonymous credentials offer mathematical guarantees of privacy, generating proofs on the client side introduces significant computational overhead and immense architectural complexity for a standard web application.
- **Final Decision:** We rejected the cryptographic approach in favor of strict application-level decoupling and database isolation. Authentication tokens are verified at the API gateway, but the feedback payload is stripped of all identifiers before being written to the database.

Decision 3: Backend Concurrency and AI Task Processing

- **Considered Alternative:** A Python-based backend (e.g., FastAPI) utilizing Celery and RabbitMQ for asynchronous processing of the Triage Agent’s AI API calls.
- **Trade-offs & Rationale:** Python has excellent AI ecosystem support. However, handling asynchronous background tasks (like our AI moderation evaluations) without blocking HTTP

threads would require deploying and maintaining a separate message broker (RabbitMQ) and worker nodes.

- **Final Decision:** We chose Go. Its native goroutines and channels allow us to spawn lightweight background workers for the AI Triage Agent directly within the monolithic backend, bypassing the need for heavy external message brokers while maintaining high throughput.

Evaluated and Rejected LLM Recommendations

During the architecture planning phase, we consulted an LLM for system design advice. We critically evaluated and rejected the following three recommendations:

1. **Recommendation: Implement a conversational chatbot interface for students to query course metrics.**

Rejection Rationale: We rejected this because the project guidelines explicitly state that bolting a chatbot onto an application is insufficient. It distracts from our core transactional workflow (the moderation queue) without adding meaningful engineering complexity.

2. **Recommendation: Adopt a distributed microservices architecture for the AI Gateway, User Auth, and Core Feed.**

Rejection Rationale: Given our baseline expectation of 100+ daily active users and our team size, a microservices architecture introduces unnecessary overhead for distributed tracing and deployment. We chose a modular monolith in Go instead.

3. **Recommendation: Use a Graph Database (e.g., Neo4j) to detect brigading networks.**

Rejection Rationale: The LLM suggested this to map relationships between spamming users. We rejected this because our Triage Agent operates on strict velocity ($V > 15$) and semantic similarity ($S > 75\%$) thresholds. We can calculate cosine similarity on embeddings efficiently in memory without the overhead of graph traversals.

3 Project Management Setup

3.1 Jira Project and Initial Backlog

The project is managed in Jira under the project key **SCRUM**:

<https://kishore200519.atlassian.net/jira/software/projects/SCRUM/summary>

The initial Jira backlog contains one issue for each requirement in the requirements traceability matrix (Table 3). The 13 functional requirements are represented as *Stories*, while the 8 non-functional requirements are represented as *Tasks*. Every requirement issue contains the corresponding requirement ID in its title, allowing the Jira issue to be directly associated with the requirement.

The 21 requirement issues are numbered from **SCRUM-1** to **SCRUM-21**. The backlog is organized according to the major functional areas of CampusEcho, covering authentication and access control, anonymous feedback, course ratings and review tracking, AI triage, appeal and moderation management, privacy and security, concurrency, and reliability and performance.

All issues are initially in the *To Do* state because the corresponding implementation work has not yet been completed. During development, issues will move through the Jira workflow according to

Jira Area	Requirements	Jira Issues
Authentication and Access Control	FR-1, FR-2	SCRUM-1, SCRUM-2
Feedback Submission and Viewing	FR-3, FR-4, FR-5	SCRUM-3 to SCRUM-5
Triage Agent	FR-6, FR-7, FR-8	SCRUM-6 to SCRUM-8
Appeals and Moderation	FR-9 to FR-13	SCRUM-9 to SCRUM-13
Privacy and Security	NFR-1, NFR-2, NFR-8	SCRUM-14, SCRUM-15, SCRUM-21
Concurrency	NFR-3	SCRUM-16
Reliability and Performance	NFR-4 to NFR-7	SCRUM-17 to SCRUM-20

Table 4: Organization of the initial Jira backlog

the progress of their implementation and verification. The planned workflow is *To Do* $\rightarrow$ *In Progress* $\rightarrow$ *In Review* $\rightarrow$ *Done*. An issue will be moved to *In Review* when its associated pull request is ready for peer review, and it will be marked *Done* only after the implementation has been reviewed, merged, and verified against its acceptance criteria.

Figure 5 shows the initial product backlog in Jira, while Figure 7 shows the issues selected for the initial sprint.

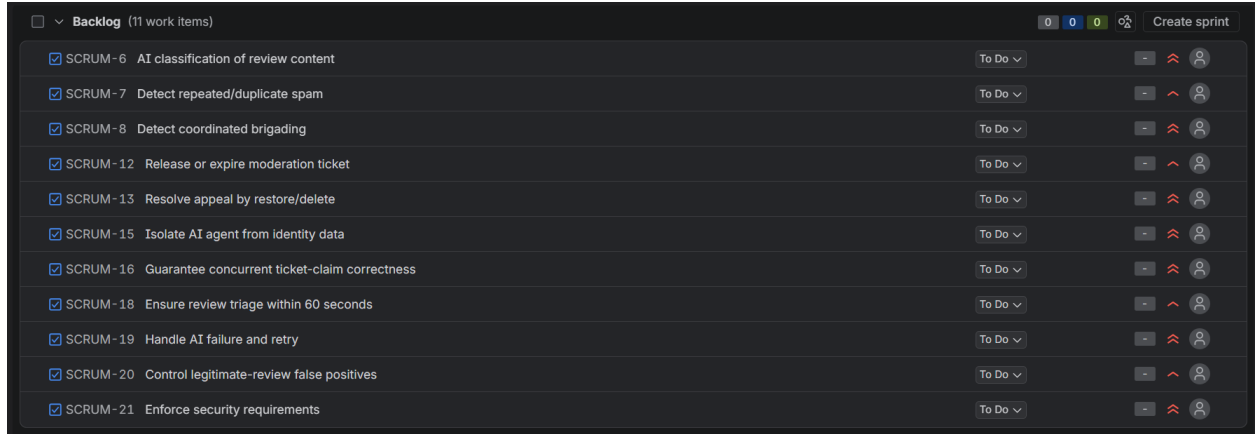


Figure 5: Initial Jira Product Backlog for CampusEcho

3.2 Jira–GitHub Integration

The Jira project is connected to the GitHub repository through the GitHub for Jira app. The connected repository is **CS455-Project**. The integration allows development activity associated with Jira issue keys to be linked to the corresponding Jira work items.

The repository is available at:

<https://github.com/Kishore200524/CS455-Project>

The team will use Jira issue keys consistently in branch names, commit messages, and pull request titles. For example, implementation work for **SCRUM-11** will use a branch following the convention **feature/SCRUM-11-atomic-ticket-claim**. This allows the corresponding GitHub development activity to be associated with the Jira issue.

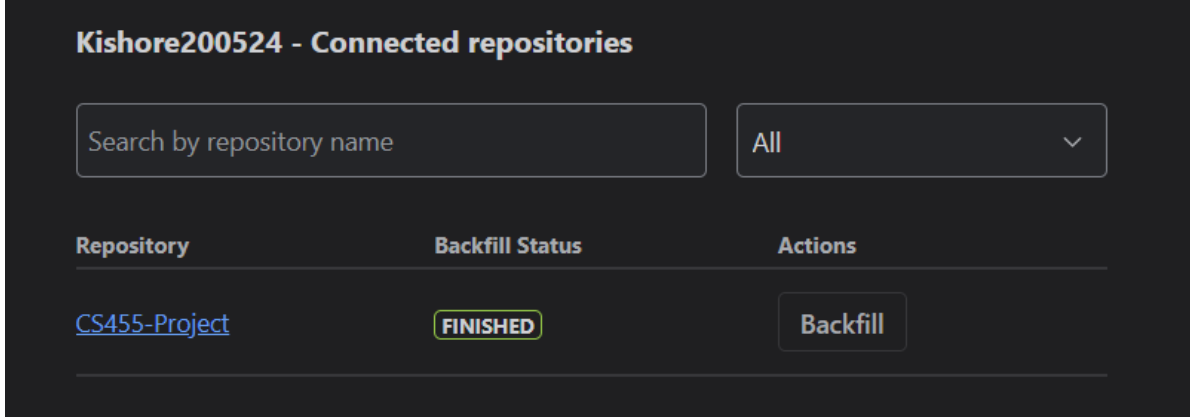


Figure 6: Jira–GitHub integration and connected CS455-Project repository

3.3 Initial Sprint Plan

Development is organised into four planned nine-day sprints. The sprint roadmap follows the dependencies in the CampusEcho review lifecycle: reviews must first be submitted before they can be triaged, and flagged reviews must exist before they can be appealed and processed by moderators.

Table 5 presents the planned sprint roadmap, while Table 6 shows the issues currently assigned to Sprint 1 in Jira.

Sprint	Dates	Goal	Requirements
1	28 Sep – 6 Oct 2026	Establish the core CampusEcho workflow for authentication, anonymous feedback submission, review tracking, appeals and moderation	FR-1 to FR-5, FR-9 to FR-11, NFR-1, NFR-4
2	7 Oct – 15 Oct 2026	Implement asynchronous AI-based review triage and associated reliability and privacy mechanisms	FR-6 to FR-8, NFR-2, NFR-5 to NFR-7
3	16 Oct – 24 Oct 2026	Complete moderation workflow and strengthen concurrency and security behaviour	FR-12, FR-13, NFR-3, NFR-8
4	25 Oct – 2 Nov 2026	Integration, refinement and verification of the complete system	Remaining work and integration activities

Table 5: Planned sprint roadmap

Sprint 1 goal. The first sprint is named **Sprint 1 – Core CampusEcho Workflow** in Jira. Its goal is to establish the foundational workflow for CampusEcho, covering authentication, anonymous review submission, review viewing and tracking, appeals, moderation-queue access, atomic ticket claiming, review anonymity, and submission responsiveness.

The sprint currently contains 10 work items in Jira. These issues remain in the *To Do* state at the start of the sprint and will be progressed through implementation, review, and verification during

the sprint.

Issue	Work
SCRUM-1	FR-1: Student authentication and moderator provisioning
SCRUM-2	FR-2: Role-based login and access control
SCRUM-3	FR-3: Anonymous review submission with a reference code
SCRUM-4	FR-4: View course ratings and published reviews
SCRUM-5	FR-5: Track review status using the reference code
SCRUM-9	FR-9: Appeal a flagged review
SCRUM-10	FR-10: View the moderation queue
SCRUM-11	FR-11: Claim a moderation ticket atomically
SCRUM-14	NFR-1: Preserve review anonymity
SCRUM-17	NFR-4: Ensure a sub-1-second submission response

Table 6: Initial Sprint 1 backlog

Figure 7 shows the 10 work items currently assigned to Sprint 1 in Jira. The remaining 11 requirement issues remain in the product backlog for subsequent sprints.

Issue	Work	Status	Assignee
SCRUM-1	Student authentication and moderator provisioning	To Do	VB
SCRUM-2	Role-based login and access control	To Do	VB
SCRUM-3	Anonymous review submission	To Do	KS
SCRUM-4	View course ratings and published reviews	To Do	MG
SCRUM-5	Track review status using reference code	To Do	MG
SCRUM-9	Appeal flagged review	To Do	RP
SCRUM-10	View moderation queue	To Do	PS
SCRUM-11	Atomic moderation-ticket claiming	To Do	PS
SCRUM-14	Preserve review anonymity	To Do	KS
SCRUM-17	Ensure sub-1-second submission response	To Do	VB

Figure 7: Initial Sprint 1 backlog in Jira

An issue is considered complete when its implementation satisfies the associated requirement and acceptance criteria, the corresponding GitHub changes have been reviewed by another team member, the pull request has been merged, and the Jira issue has been updated to its final status. Incomplete issues remain open and are carried forward rather than being marked as done artificially.

4 AI Engineering Log for Requirements and Architecture Activities

LLMs were used during the requirements engineering and architecture design phases of CampusEcho to assist with requirement refinement, architectural alternatives, workflow design, and identification of potential engineering risks. The outputs generated by the LLM were treated as design suggestions rather than authoritative solutions.

Each major AI-assisted activity was reviewed by the team. The team evaluated the generated recommendations for correctness, feasibility, privacy, concurrency, failure handling, and consistency

with the project requirements.

4.1 AI-Assisted Requirements Refinement

4.1.1 Objective

The first use of an LLM was to review the initial CampusEcho requirements and identify ambiguity, missing edge cases, and requirements that could be difficult to verify through testing.

The objective was to ensure that the requirements were:

- Unambiguous.
- Testable.
- Consistent with the proposed workflow.
- Suitable for conversion into Jira issues.
- Explicit about system behaviour during failures and concurrent access

4.1.2 Prompt / Interaction

The team provided the initial CampusEcho project description, user stories, functional requirements, and non-functional requirements to the LLM and asked it to review the requirements from a software-engineering perspective.

Prompt:

Review the following requirements for CampusEcho, an anonymous course-feedback and moderation system. Identify ambiguous, incomplete, conflicting, or difficult-to-test requirements. Suggest specific improvements, especially for anonymity, AI triage, concurrent moderation-ticket claiming, failure handling, and response-time requirements. Do not introduce unnecessary features.

The team provided the complete CampusEcho requirements, including the user stories, functional requirements, non-functional requirements, and system constraints, to the LLM. The LLM was then asked to generate the component and sequence diagrams based on these specifications.

Prompts:

Using the CampusEcho requirements given above, create a component diagram for our system. Show the main components, how they communicate with each other, and how we keep student identity information separate from review data.

Using the CampusEcho requirements given above, create a sequence diagram for the asynchronous AI triage process. Show the student submitting feedback, the background AI processing, spam and brigading detection, the publish/flag decision, and what happens if the AI service fails.

Using the CampusEcho requirements given above, create a sequence diagram for concurrent moderation-ticket claiming. Show two moderators trying to claim the same ticket at the same time, how the ticket is claimed atomically, and how only one moderator succeeds.

Using the CampusEcho requirements given above, create a sequence diagram for the appeal resolution and ticket lifecycle. Show ticket creation, claiming, restoring or deleting the review, releasing the ticket, and what happens when the 30-minute limit is reached.

4.1.3 LLM Output

The LLM identified several areas requiring clarification, including:

- The distinction between anonymous review data and authentication data.
- The behaviour when AI triage fails or times out.
- The exact condition under which a moderation ticket can be claimed.
- The handling of tickets after the claim timeout.
- The need to define the brigading threshold precisely.
- The requirement that review submission should not depend on AI-processing latency.

4.1.4 Team Evaluation

The team considered these recommendations useful because they directly affected testability and system behaviour.

However, the team did not adopt recommendations simply because they were generated by the LLM. Each suggestion was compared against the intended CampusEcho workflow and project constraints.

The team particularly adopted the recommendation to explicitly define failure behaviour for AI triage and to make the concurrent ticket-claiming condition deterministic.

4.1.5 Changes Made

The requirements were refined to explicitly specify:

- Reviews initially enter the Submitted state.
- AI triage occurs asynchronously.
- AI failure or timeout leaves the review in Submitted.
- Failed AI processing can be retried.
- A review is not published without successful triage.
- Only one moderator can successfully claim a particular moderation ticket.
- Tickets automatically expire after 30 minutes if not resolved.
- Brigading detection uses explicit review-volume and semantic-similarity thresholds.

4.1.6 Final Decision

The LLM output was modified and partially accepted.

The team retained recommendations that improved requirement precision and testability while rejecting suggestions that introduced unnecessary functionality or complexity.

4.2 AI-Assisted Architecture Generation

4.2.1 Objective

The team used an LLM to generate initial architecture alternatives for CampusEcho based on the functional requirements, concurrency requirements, anonymity constraints, and planned AI integration.

The objective was to explore multiple architectural approaches before selecting the final design.

4.2.2 Prompt / Interaction

Prompt:

Design a software architecture for CampusEcho, an anonymous course-feedback platform with Student and Moderator roles. The system must support asynchronous AI-based review triage, spam and brigading detection, appeals, a shared moderation-ticket queue, and concurrent ticket claiming. Reviews must remain anonymous and the AI component must not access user identity information. Suggest suitable components, database structure, concurrency mechanisms, AI integration, failure handling, and relevant design patterns. Also provide alternative architectures and their trade-offs.

4.2.3 LLM Output

The LLM proposed several possible architectures, including:

1. A modular monolithic architecture with asynchronous AI workers.
2. A microservices architecture separating authentication, reviews, moderation, & AI processing.
3. A distributed architecture using Redis for ticket locking.
4. A graph database approach for detecting coordinated review activity.

4.2.4 Team Evaluation

The team evaluated the alternatives based on:

- Implementation complexity.
- Deployment complexity.
- Concurrency correctness.
- Privacy requirements.
- Failure handling.
- Scalability.
- Number of external dependencies.
- Suitability for the project timeline.
- Ease of testing and debugging.

The team concluded that a fully distributed architecture would introduce additional complexity that was not necessary for the initial system.

4.2.5 Final Architecture

The selected architecture is a **modular monolithic architecture** implemented using Go, with asynchronous background processing for AI-based moderation. The architecture was chosen to provide clear separation of responsibilities while avoiding the additional deployment and operational complexity of a fully distributed microservices system.

The major components of the selected architecture are:

1. **React.js + Tailwind CSS Frontend:** Provides separate dashboards and interfaces for students and moderators. Students can submit feedback, view published feedback and initiate appeals, while moderators can access and resolve moderation tickets.
2. **Go Backend:** Acts as the central application layer and coordinates authentication, feedback submission, moderation workflows, appeals, and access control. The backend follows a modular structure so that different responsibilities remain logically separated within the monolithic application.
3. **MongoDB:** Stores feedback, moderation tickets, and other application data. MongoDB atomic operations are used for the moderation-ticket claiming mechanism so that concurrent moderators cannot acquire the same ticket.
4. **Asynchronous AI Worker Pool:** Newly submitted feedback is processed asynchronously using Go goroutines and worker pools. This prevents AI processing from blocking the feedback submission workflow.
5. **AI Triage Agent:** The agent analyzes individual feedback for hate speech or abuse and performs macro-level analysis for spam and coordinated brigading. It uses the velocity and semantic-similarity metrics specified in the project requirements to determine whether a batch should be flagged.
6. **AI API Gateway:** The AI triage agent interacts with the selected LLM service (Claude/AWS Bedrock) through an internal API boundary. The agent is prevented from accessing identity-mapping information.
7. **Moderation Queue:** Appeals generated for flagged feedback are placed into a shared moderation-ticket pool. Moderators pull tickets from this queue and attempt to claim them using an atomic database operation. Once claimed, the ticket is locked to exactly one moderator for processing.
8. **Authentication and Identity Separation:** Authentication information is kept logically separate from the feedback payload so that anonymous feedback can be processed without exposing the submitter's identity to moderators or the AI triage agent.

The resulting high-level workflow is:

Student → Go Backend → MongoDB → Asynchronous AI Worker → AI Triage Agent

For feedback that is flagged and subsequently appealed, the workflow continues through the shared moderation queue:

Flagged Feedback → Appeal Ticket → Moderation Queue → Atomic Ticket Claim → Moderator Review → Restore / Delete

This architecture therefore preserves the required anonymity boundary, supports asynchronous AI processing, and directly addresses the project's central concurrency challenge involving multiple

moderators attempting to claim the same moderation ticket.